

Improving Playlist Recommendation Systems by Finetuning BERT Models on Clustered Data

Siddhant Mohanty

Abstract—Music playlist titles are generally created with a specific theme or genre of music in mind. This allows creators to share and catalog their preferred music in an easily accessible and understandable way to an outside user. This study replicates work done previously to recommend tracks that match the semantic structure of a playlist title using LLMs and the Spotify Million Playlist Dataset (MPD), and makes significant improvements on the results of these researchers. The initial authors clustered with K-means, which imposes strong assumptions on the data; namely that the clusters are spherical, of equal volume, and that cluster membership is hard. This study applies two alternative strategies; fuzzy k-means is used on raw playlist embeddings and HDBSCAN on a low-dimensional representation of the data in 15 dimensions, after which spurious clusters are pruned with a new LSH based algorithm. Whereas the method the original researchers use obtains roughly 20% accuracy on the test dataset, the methods introduced here reach lower overall training and validation losses and hit 60% accuracy.

I. INTRODUCTION

Recommendation systems are used by customer-facing enterprises throughout the world to drive engagement and increase sales and retention. Companies like Netflix, YouTube and Spotify use these systems despite not selling a product in a conventional sense. The goal of these entities is not to drive sales directly by recommending products that users are likely to purchase, but instead to keep users engaging with the application or software for as long as possible in order to maximize ad exposure and streaming retention rates.

Free streamed media presents a unique challenge for recommendation systems; Unlike products, useful metrics such as the price of a song or its absolute, 'true' genre either do not exist or are impossible to evaluate objectively.

In an attempt to address this problem, Spotify released the Million Playlists Dataset (MPD). This dataset contains user-generated playlists and is comprised of playlist titles, song runtimes, and the number and names of each track within each playlist. With user-generated playlists, some of the work has been offloaded; users have created a list of songs they enjoy, and titled the playlist something identifiable. This presents an interesting question to the field of language modeling in the recommender system space: Does the semantic makeup of a playlist have any similarities with the title of the playlist itself? In other words, are there generalizable qualities of playlist titles that are reflected in the songs contained within?

In general, we expect this to be true; a playlist titled "Workout Beats" should contain energetic, upbeat music, whereas "Lo-Fi Relaxation Vibes" might ostensibly contain low-fidelity, slow-tempo songs. In this sense, playlist titles are a powerful source of information for recommendation systems,

especially if the title lacks specific terminology that is only meaningful to the creator.

In this study, I create an end-to-end pipeline that clusters playlist representations, using three clustering algorithms: k-means, fuzzy k-means, and HDBSCAN [1]. Clustering is performed on mean-pooled track embeddings and clusters here are proxies for the broad categories of music that make up the data in the MPD. Spurious clusters are then pruned using a Locality-Sensitive Hash (LSH) algorithm before finetuning a Sentence-BERT model on the resulting cluster labels [2]. This finetuned model is used when making inferences to create an embedding of a playlist title (instead of using the original BERT model that generated the track embeddings) to retrieve the most similar tracks.

This study is relevant to web intelligence in a variety of ways. It focuses on improving how music platforms recommend tracks to users based on playlist titles using information retrieval and recommender systems. It seeks to provide personalized recommendations to users without tedious manual searching. Web Intelligence relies on computers to understand human behavior and language to model connections between items, users, and products. The playlist similarity and clustering algorithms described here can be generalized to many applications, where LLMs such as Sentence-BERT can be used to generate meaningful vectorizations of the data, circumventing the often laborious process of information retrieval that is used to build user and item profiles. This process also avoids the cold-start problem because recommendations are made simply from a playlist title entered by a user. No additional information is required to present tracks to a user.

The remainder of this paper is structured as follows: Section II discusses prior work. Section III describes the pipeline in detail and discusses necessary background. Section IV presents and analyzes experimental results. Section V concludes the paper and discusses future avenues of study for this problem.

II. RELATED WORK

The use of language models to generate song recommendations given a playlist name is an experiment that has been conducted before [3]. In 2025, an experiment was conducted using the Sentence-BERT model all-MiniLM-L6-v2 to create embeddings for each track in the MPD playlists, which were then clustered.

The clustering method of choice was K-Means, where the optimal clustering arrangement S is found by minimizing the within-cluster sum of squares:

$$\operatorname{argmin}_S \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2 \quad (1)$$

Where μ_i is the centroid of the points in cluster S_i :

$$\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x \quad (2)$$

This clustering method is well-known for its speed and as a strong starting point for other clustering algorithms. However, it should be noted that this algorithm has significant weaknesses that are not addressed. Namely, the assumption of spherical clusters of equal volume and that playlists must belong exclusively to a single cluster.

The original researchers tested a range of cluster counts and settled on 200 clusters after studying the elbow plot of distortion shown in figure 1. Though this figure suggests roughly 50 clusters, they choose to use 200 because it minimizes distortion. After clustering, a Sentence-BERT model was finetuned on the resulting cluster labels and used to create embeddings of the playlist titles. The playlist titles were ignored during the clustering process, but used during inference. In this way, playlists can be populated using the tracks they are most similar to. As an additional step, the researchers use the open source LLM LLaMA-2-7B-32K to re-rank the tracks through text analysis. This study replicates the majority of these steps, but due to time constraints, it does not perform the LLM reranking; LLaMa-2-7B-32K is a very large model and requires many GPU hours to re-rank the data.

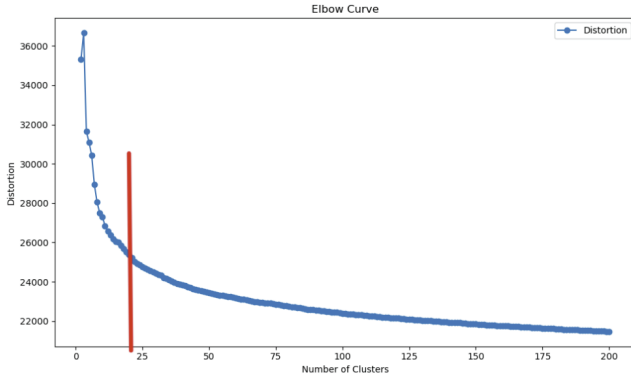


Fig. 1. Elbow plot depicting cluster distortion as a function of k from [3].

The K-means method achieves approximately 20% accuracy on the test dataset. This study proposes two clustering alternatives that challenge the two strongest assumptions made by K-means: (1) that clusters are spherical and of equal volume, and (2) that cluster membership is exclusive. In the context of music, neither assumption is well-justified. Genres are adjacent and overlapping, and the embedding space of track titles is highly likely to be non-spherical and non-convex. For example, baroque and classical piano music are closer than country music and Mongolian throat singing; pop songs

today sometimes feature rap artists, though these styles of music are quite different. Clustered tracks should reflect these peculiarities of the data.

III. METHODS

The pipeline consists of four stages: preprocessing, clustering, pruning, and finetuning.

A. Preprocessing

The training process begins with data preprocessing. The MPD dataset is a large set of JSON slices containing the information about each playlist. This includes the playlist title, and the tracks contained within, which are the most important features for the analysis in this study, but also include additional data, such as whether the playlist was collaborative or not, and the total runtime of each playlist alongside the runtime of each track.

The data was distilled into three csv files to resemble a database structure with multiple tables that can access each others elements using primary and foreign keys:

- A file containing the identifying information of all the playlists. This includes the ids, titles, number of tracks, and so on, but omits the tracks themselves.
- A file linking each track to the playlists it appears in based on the unique ids of each.
- A file containing the unique tracks, their runtimes, as well as their titles, artists and album names.

Text must be represented numerically for computations to be performed on it. Convention is to convert text into vectors that capture some latent qualities of the constituent words. Options such as one-hot encoding are unsuitable as they create vectors with dimension equal to the number of unique tracks. These vectors are sparse and difficult to cluster or compare due to the curse of dimensionality. Instead, in this study, I use the all-MiniLM-L6-v2 Sentence-BERT encoding model to generate 768-dimensional embeddings for each track.

In addition, I represent each playlist as a mean-pooling of the tracks that are contained within:

$$e_P = \sum_{t \in P} e_t \quad (3)$$

This results in a set of one million playlist embeddings that can be clustered.

B. Clustering

Music is grouped by named labels created by humans; we call these genres. Clustering on the generated embeddings results in similar groups, with a key distinction. Songs are categorized into human-made genres based on their musical style and instrument composition. Because this information is lacking in the MPD, a key assumption we make is that track titles contain some latent semantic qualities that align with the genre of the song.

1) *K-Means (Replication)*: Following the original researchers, K-Means clustering is applied to the 768-dimensional playlist embeddings with $k = 200$ clusters.

When using this approach, some clusters are very large in diameter; the playlists contained within lack semantic homogeneity. To prevent these large and scattered clusters from biasing the training process, the original researchers remove them by deleting clusters where the modal playlist appears in less than 2% of the rows in those clusters. This process deletes around 400,000 playlists and 42 clusters.

I also apply two different clustering methods compared to the original researchers, which each explore other, less strict assumptions.

In the realm of music, we might actually expect clusters of non-spherical shapes to be the norm. Additionally, because of the adjacency of music genres and the prevalence of genre mixing in today’s music, it seems reasonable that a track should have membership in multiple clusters.

2) *Fuzzy K-Means*: Fuzzy clustering seeks to create a membership vector for each observation in the dataset, relaxing the hard-assignment assumption of K-Means. The central idea is that an observation can be a partial member of every cluster. The degree of membership is used to interpret where the observation belongs. To this end, we seek the optimal cluster arrangement S , where the optimization function being minimized is

$$\operatorname{argmin}_S \sum_{i=1}^k \sum_{x \in S_i} \mathbf{u}_i^v \|\mathbf{x} - \mu_i\|^2 \quad (4)$$

Here, $\mathbf{u}_i^v$ represents the membership vector, where each element is raised to the power v , which is called the fuzzifier; the i^{th} element of this vector is the membership of the observation $\mathbf{x}$ in cluster i . The fuzzifier in Fuzzy K-Means is set to 2 by default. This algorithm tackles the partial membership problem in this dataset. The memberships are constrained to add up to 1 for a particular observation.

Unlike the original authors, I perform fuzzy clustering using the number of clusters recommended by the elbow plot, 50. Fuzzy clustering also discards clusters that are devoid of observations with principal memberships. In other words, if a cluster c exists such that no observation has maximum membership in c , it is ignored.

3) *HDBSCAN*: HDBSCAN is a density-based clustering algorithm that is capable of fitting clusters of arbitrary shapes. It is capable of automatically isolating points as noise (cluster label = -1). It is a stronger, more optimized version of DBSCAN, which the original researchers attempted to use, but were unsuccessful with. I suspect this is due to the dimensionality of the dataset. To combat this, I apply a Uniform Manifold Approximation and Projection (UMAP) transformation to the data [4].

UMAP seeks to create a fuzzy topological representation of a high-dimensional space. It identifies a non-euclidean manifold that the data can be projected onto, serving as a

UMAP 2D — Mean-Pooled Playlist Embeddings (n=20,000 sample)

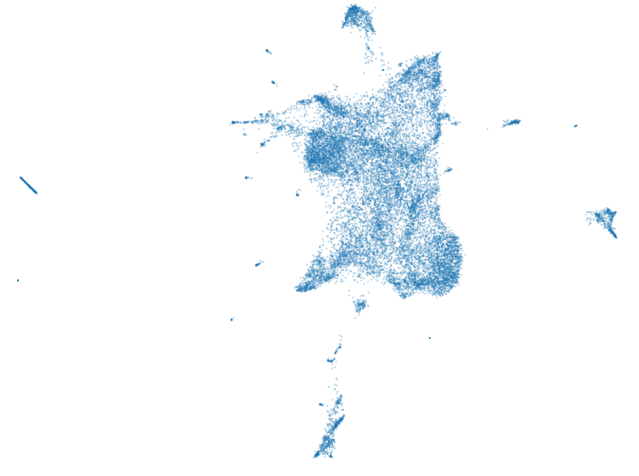


Fig. 2. A sample of 20,000 playlist embeddings from the UMAP transformed space, projected from 15 dimensions to 2 for interpretability

low-dimensional representation of the data that retains this topological structure.

UMAP is run with the following configuration: $n_components \in \{10, 15, 20\}$, $n_neighbors = 15$, $min_dist = 0.0$, and $metric = cosine$. The reducer is fit on a random subsample of 100,000 playlists and then applied to the full dataset. This sample size balances the accuracy and resolution of the manifold with efficient computation. Performing UMAP on the entire dataset is prohibitive. To select the best dimensionality, trustworthiness and continuity are evaluated on 5,000 test points that were not included in the original sample of 100,000 playlists, as shown in Table I. The 15-dimensional projection is selected as it achieves the lowest noise fraction while maintaining comparable trustworthiness, though the choice of dimension is somewhat arbitrary; the metrics are all very close. A sample of the transformed data is shown (projected into 2 dimensions for visualization purposes) in Figure 2

TABLE I
UMAP DIMENSIONALITY SELECTION

UMAP Dims	Trustworthiness	Continuity	HDBSCAN Noise %
10	0.8339	0.9425	2.14%
15	0.8332	0.9420	2.00%
20	0.8331	0.9420	2.12%

A grid search was performed over a set of HDBSCAN parameters in the pursuit of optimal clustering. I vary the $min_cluster_size$ values between [100, 200, 500, 1000] and the $epsilon$ s between [0, 5, 10, 15, 20, 25, 30]. $epsilon$ controls the distance threshold at which nearby clusters are merged. The top five configurations are tabulated in table II.

Based on these results, I ran HDBSCAN with $min_cluster_size$ set to 100 and $epsilon$ set to 10. This gave 3 clusters with memberships described in table III.

TABLE II
TOP 5 CLUSTERING HYPERPARAMETER CONFIGURATIONS (SORTED BY RELATIVE VALIDITY)

min_cluster_size	cluster_selection_epsilon	n_clusters	noise_frac	relative_validity
100	10	3	0.0001	0.1831
100	5	12	0.0001	0.1785
200	5	9	0.0007	0.1780
500	5	5	0.0020	0.1769
1000	5	4	0.0029	0.1761

TABLE III
CLUSTER SIZE SUMMARY FOR THE BEST CONFIGURATION

Index	Cluster ID	Playlist Count
0	2	999,382
1	0	371
2	1	173

It is evident that the grid search fails to capture the true structure of the data that is apparent through visual inspection, instead grouping almost all points into the same cluster. I opted to discard the results of this search and instead proceed with default HDBSCAN parameters with `min_cluster_size = 200`, `min_samples = 10`, and `cluster_selection_method = eom`.

C. LSH Cluster Cleaning

The original researchers pruned a cluster if its most common playlist title appeared in fewer than 2% of the cluster’s rows. This approach has a major shortcoming; exact title matching ignores semantically similar titles (e.g., “Rock music” and “Rock ’n Roll” are treated as distinct). This method was likely used because with one million playlists, computing pairwise similarity for all titles would be computationally infeasible with time complexity $O(N^2)$.

This study replaces this pruning style with an LSH-based approach. Playlist embeddings are hashed into 100 buckets. Embeddings with $\geq 70\%$ cosine similarity are likely to collide into the same bucket. For each cluster, if the largest bucket contains fewer than 2% of the cluster’s playlists, the cluster is dropped as semantically incoherent. This approach is more robust to minor variations in playlist title wording while retaining the same computational scalability as the original pruning strategy.

D. Finetuning Sentence BERT

After cluster cleaning is complete, the surviving cluster labels are used as training data to finetune the same Sentence BERT model used to create playlist embeddings. The clusters are split into a training set containing 90% of the data and a validation set containing the remaining 10%. Training uses the AdamW optimizer with a learning rate of 2×10^{-5} , weight decay of 0.01, and a linear warmup over 100 steps. A batch size of 64 is used. Early stopping with patience 5 is applied to halt training when validation accuracy ceases to improve.

Two finetuning strategies are evaluated:

Soft cross-entropy on fuzzy labels. The membership vector $\mathbf{u}_i$ for each playlist is used directly as a soft target distribution in a KL-divergence cross-entropy loss. This approach is applied to the FKM clusters (50 cluster target, pre-pruning). Because this step was very time-consuming (roughly 4 times as long as can be seen in the notebook runtime) and learned poorly, I pivoted to the following strategy.

Hard cross-entropy on hard labels. The argmax cluster assignment is used as a discrete label in standard cross-entropy loss. This is applied to (a) the LSH-pruned FKM clusters and (b) the HDBSCAN clusters (both with and without LSH pruning).

To replicate the baseline, I only trained K-Means for 5 epochs. The number of training epochs for Fuzzy K-Means and HDBSCAN finetuning was set to a maximum of 20 to avoid overfitting and reduce computation time. In some cases, if stopping criteria were met, training was halted early, resulting in fewer epochs.

IV. RESULTS

A. Clustering

Fuzzy K-Means. FKM with a target of 50 clusters based on the elbow plot populated 35 distinct clusters across the full dataset of one million playlists, naturally discarding 15 clusters due to weak memberships. After LSH pruning, 10 of these 35 clusters were dropped as incoherent, retaining 25 clusters. Figure 3 shows a t-SNE visualization of the unpruned FKM clusters. Figure 4 shows the same visualization after LSH pruning.

HDBSCAN. Applied to the 15-dimensional UMAP reduction, HDBSCAN identified 133 real clusters alongside a noise cluster. Approximately 66% of the one million playlists were labeled as noise and excluded, leaving 339,800 playlists across 133 clusters for finetuning. After applying LSH pruning, only 3 clusters were dropped, retaining 338,246 playlists. This result indicates that HDBSCAN already isolates noise points by design; the LSH pruning step barely contributes. Clustering results are shown in figure 5.

B. Finetuning

Soft cross-entropy on FKM fuzzy labels. When trained with the fuzzy membership vector as a soft target distribution, the training and validation losses remained flat at approximately 3.91 across all 20 epochs. This indicates that the model did not learn to reduce uncertainty; the fuzzy targets may have

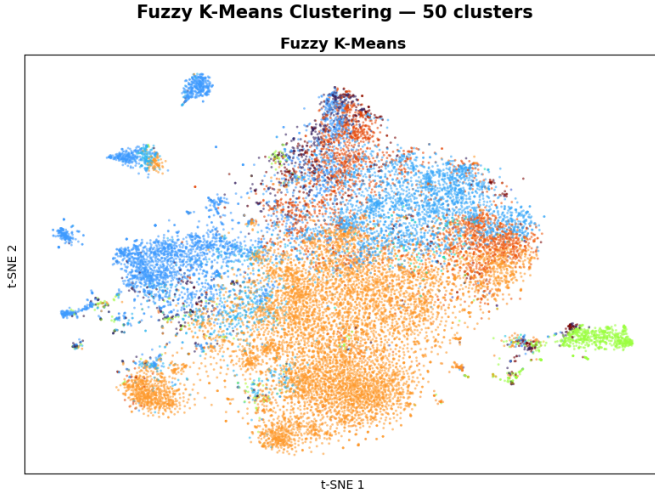


Fig. 3. t-SNE projection of FKM cluster assignments (35 clusters, pre-pruning).

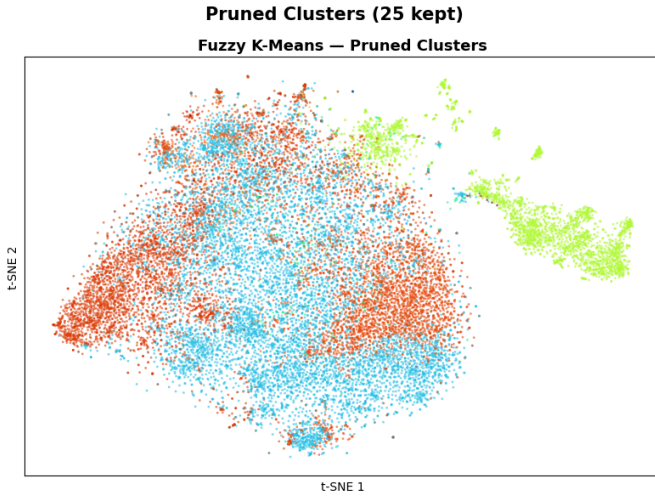


Fig. 4. t-SNE projection of FKM cluster assignments after LSH pruning (25 clusters retained).

been too diffuse to provide a useful training signal. The model is preserved in its near-pretrained state as a result.

Hard cross-entropy on LSH-pruned FKM clusters. Using hard argmax cluster assignments after pruning, the model trained effectively, as can be seen in Table IV. The training loss decreases steadily, while the validation loss reaches its minimum around epoch 7 (1.171) before rising, indicating overfitting. The best validation accuracy of 54.8% is recorded at epoch 7. For reference, the K-means baseline achieves approximately 20% accuracy over 200 clusters after 5 epochs.

Hard cross-entropy on HDBSCAN clusters. HDBSCAN finetuning started with a higher initial loss than fuzzy k-means. However, it converged to a substantially higher accuracy, reaching a peak of 60.3% at epoch 6 and early-stopping at epoch 11. Table V shows the per-epoch metrics for both the unpruned and LSH-pruned HDBSCAN runs.

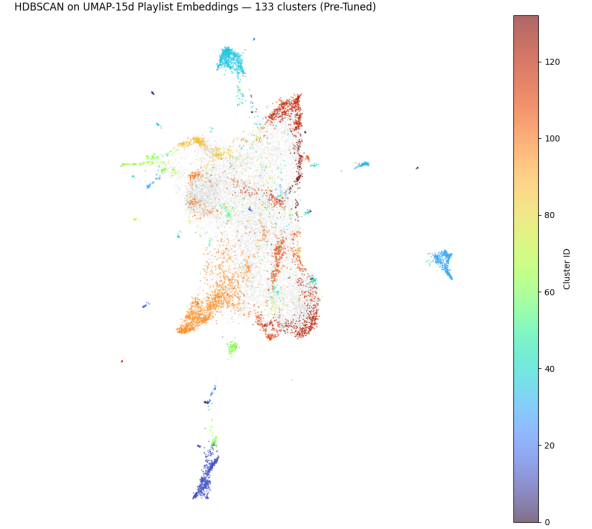


Fig. 5. HDBSCAN performed with default parameters on the UMAP data. The density based structure is respected and retained here and many individual clusters can be seen.

TABLE IV
FKM HARD FINETUNING — PER-EPOCH METRICS (25 CLUSTERS)

Epoch	Train Loss	Val Loss	Accuracy
1	1.2797	1.1806	0.5437
2	1.1735	1.1823	0.5381
3	1.1601	1.1685	0.5428
4	1.1479	1.1682	0.5450
5	1.1379	1.1796	0.5332
6	1.1325	1.1703	0.5406
7	1.1252	1.1708	0.5477
8	1.1170	1.1697	0.5474
9	1.1099	1.1701	0.5457
10	1.1033	1.1740	0.5450
14	1.0827	1.2003	0.5431
20	1.0577	1.2209	0.5421

The pruned and unpruned HDBSCAN models achieve nearly identical accuracy (60.3% vs. 60.2%), confirming empirically that LSH pruning adds negligible benefit when applied after HDBSCAN, which inherently classifies some playlists as noise. Figures 6–10 show training and validation loss curves for all models, including the K-means baseline.

Here, accuracy is defined as how often the cluster label predicted by the BERT model matches the cluster that was assigned by the clustering algorithm.

C. Downstream Recommendation Evaluation

All four finetuned models are evaluated on a 22-playlist test set using the recommendation pipeline from the original paper. Each playlist actually exists in the dataset and contains a list of tracks. The true tracks are hidden from the recommendation system and the playlist title is transformed into an embedding using the finetuned models. For each query playlist title, the top-50 most similar playlists are retrieved by cosine similarity of their title embeddings, and the top-10 unique tracks by occurrence frequency from those playlists are returned as

TABLE V
HDBSCAN HARD FINETUNING — PER-EPOCH METRICS

Epoch	Unpruned (133 clusters)		Pruned (130 clusters)	
	Val Loss	Accuracy	Val Loss	Accuracy
1	2.3134	0.5906	2.3141	0.5873
2	2.2664	0.5953	2.2704	0.5941
3	2.2486	0.6007	2.2497	0.5992
4	2.2381	0.6020	2.2432	0.5985
5	2.2386	0.6009	2.2367	0.6010
6	2.2366	0.6027	2.2421	0.6012
7	2.2353	0.6024	2.2391	0.5997
8	2.2362	0.6025	2.2513	0.5988
9	2.2470	0.5994	2.2431	0.6016
10	2.2367	0.6021	2.2477	0.6007
11	2.2481	0.5998	2.2518	0.6009

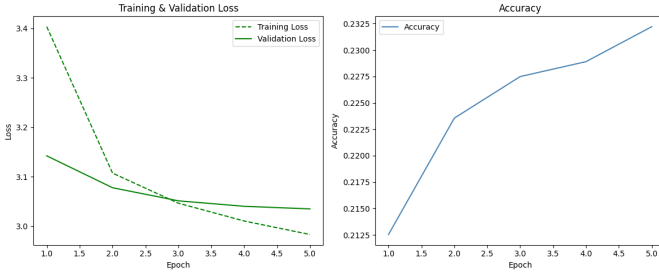


Fig. 6. K-Means

recommendations. These are compared to the ground truth, which is the set of actual tracks in the test playlist. Table VI summarizes the results.

The metrics used are **precision**, which measures the fraction of the 10 recommended items that actually exist in the original query playlist, **recall**, which measures the number of correct tracks recommended over the total number of tracks in the playlist, **mean reciprocal rank** (MRR), which measures how high up in the 10-track recommendation list the first correct recommendation appears, **r-precision**, which is a tailored precision threshold that takes into account the size of each specific playlist, and **normalized discounted cumulative gain** (NDCG) which is an alternative to MRR that is somewhat more relaxed. NDCG rewards the model based on putting correct recommendations in general towards the top of the list.

HDBSCAN achieves the best precision (0.2227), recall (0.0432), and r-precision (0.0576) indicating strong performance on exact track retrieval. FKM Fuzzy records the best MRR (0.4860) and NDCG (0.4966), suggesting its embeddings are better at ranking relevant tracks highly even when exact track retrieval performs more poorly.

The strong performance of the FKM Fuzzy model is interesting given that its classification training did not converge. Because the soft cross-entropy loss provided no useful learning behavior, the model’s weights remain close to the pretrained BERT state. The MRR and NDCG scores suggests that the pretrained BERT embeddings already carry strong semantic signal for playlist ranking, and that additional finetuning on

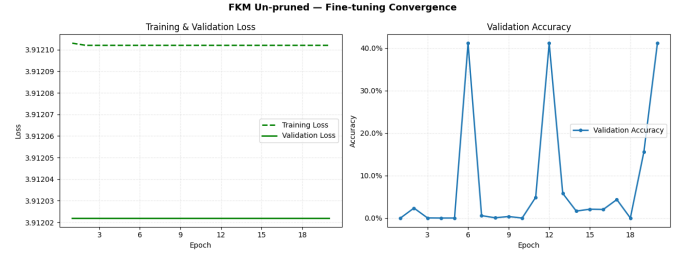


Fig. 7. Unpruned Fuzzy K-Means

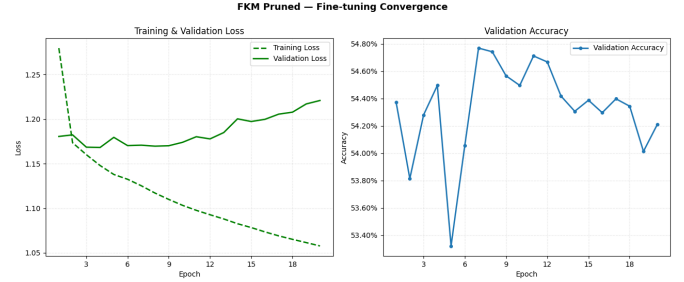


Fig. 8. Pruned Fuzzy K-Means

discrete cluster labels may actually harm ranking to improve classification accuracy.

Applying LSH pruning to HDBSCAN clusters does not have a significant impact, indicating that the noise classification capabilities of HDBSCAN are already sufficient in pruning applications.

D. Interpretation

I believe fuzzy clustering suffers when finetuning the BERT model in this dataset because of the large dimensionality. In high-dimensional spaces, the distance between observations is nearly identical because the sheer number of dimensions along which observations are compared counteracts any deviations in individual dimensions. As a result, Fuzzy k-means places most cluster centroids in the middle of the data, resulting in poorly defined memberships throughout that are not conducive to training. Pruned Fuzzy K-means actually displays a similar problem, where the validation loss does not decrease over the epochs, even though the training loss does. I suspect a similar root cause. HDBSCAN finetuning adheres more strongly to expected behavior, showing a training and validation loss that appear to decrease continuously.

It should be noted that despite the interesting learning behavior of fuzzy clustering on the finetuning step, the accuracy measure in both pruned fuzzy k-means is quite high. This indicates that LSH cluster pruning is a strong candidate for spurious cluster removal.

The performance of both methods on the recommendation evaluations is also good, indicating (at least in the case of unpruned fuzzy k-means) that pre-tuned BERT is also a good recommendation system in its own right.

TABLE VI
MODEL EVALUATION METRICS

Model	Precision@10	Recall@10	MRR@10	R-Precision	NDCG
FKM Fuzzy	0.2182	0.0392	0.4860	0.0526	0.4966
FKM Pruned	0.2091	0.0386	0.3641	0.0515	0.4437
HDBSCAN	0.2227	0.0432	0.3924	0.0576	0.4206
HDBSCAN Pruned	0.1818	0.0339	0.3455	0.0465	0.4383
Original Paper (K-means)	0.1793	0.0382	0.3254	0.0496	0.3740

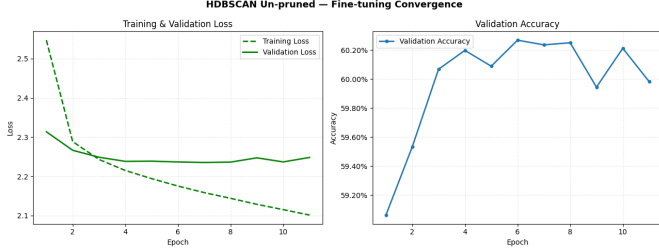


Fig. 9. Unpruned HDBSCAN

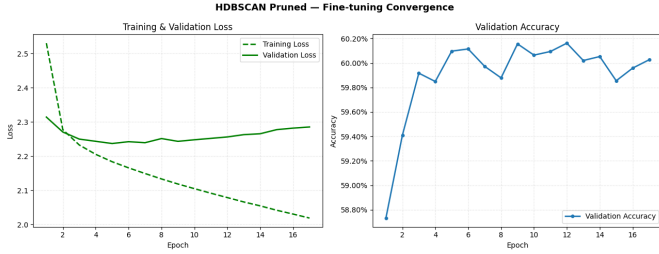


Fig. 10. Pruned HDBSCAN

Notably, HDBSCAN unpruned and both FKM models uniformly outperform the results of the original paper on all model evaluation metrics.

V. CONCLUSION AND FUTURE WORK

This paper explores alternative clustering paradigms to improve the recovery of semantic information from playlist titles and tracks. The empirical evidence provided in this study indicates that the assumptions imposed by the original authors were indeed too strict, and that both fuzzy k-means and HDSCAN on reduced-dimension data perform better. I recommend HDBSCAN as the optimal clustering algorithm out of those tested. Because HDBSCAN already isolates noise and retains coherence between training behavior and recommendation results when compared to fuzzy k-means, its results are more explainable and interpretable. However, the power of the new contribution applied herein, the LSH cluster-pruning, should not be understated. It was able to induce some learning behavior into the previously stagnant unpruned fuzzy k-means and decreased training and validation loss significantly.

The results are promising for continued work with recommendation systems. I have identified three areas for additional study and improvements:

A. Fuzzy Clustering Optimization

The fuzzy clustering algorithm failed to learn well. Future work should verify whether better initialization, different fuzzifier values, random starts, or dimensionality reduction can be applied to the data before fuzzy k-means is applied to reduce problems associated with the curse of dimensionality.

B. Word Embedding Models

The original researchers use Sentence-BERT to create embeddings for each track and each playlist. In the case of single word tracks or playlists, this is no better than a word-by-word embedding model. Given that an LLM is used for re-ranking later on, I am interested in comparing the performance of a word embedding model to the output that these researchers obtained.

The original authors used fastText, but I plan to use tiktoken GPT-2 embeddings.

C. Alternative LLMs

LLaMA-2-7B-32K is a powerful open source model used for the re-ranking process, but is likely overkill for this application. With a 32,000 token long context limit and 7 billion parameters, the model takes up a lot of room on disk before predictive and generative processing. It also requires a specialized GPU with high VRAM to hold the model and run predictions. The entire pipeline prior to LLM re-ranking is compatible with CPU-only use. Smaller language models will likely create a more portable system without sacrificing too much recommendation power.

I will test smaller open source language models such as Qwen3.5-0.8B [5] or Gemma-3n-E2B-IT [6]. Qwen 3.5 allows a longer context length ($\sim 260,000$ tokens), but is much smaller in memory with only 800 million parameters. Gemma-3n-E2B-IT uses a 32,000 token context but reduces the parameter space by more than 65%, down to 2 billion. These small models would make the full pipeline accessible on consumer GPUs.

REFERENCES

- [1] C. Malzer and M. Baum, “A hybrid approach to hierarchical density-based cluster selection,” in *2020 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. IEEE, 2020, p. 223–228. [Online]. Available: <http://dx.doi.org/10.1109/MFI49285.2020.9235263>
- [2] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [3] E. Charolois-Pasqua, E. Vellard, Y. Rebboud, P. Lisena, and R. Troncy, “A language model-based playlist generation recommender system,” *Proceedings of the Nineteenth ACM Conference on Recommender Systems*, p. 1–11, Sep 2025.
- [4] L. McInnes, J. Healy, and J. Melville, “Umap: Uniform manifold approximation and projection for dimension reduction,” 2020. [Online]. Available: <https://arxiv.org/abs/1802.03426>
- [5] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [6] G. Team, “Gemma 3n,” 2025. [Online]. Available: <https://ai.google.dev/gemma/docs/gemma-3n>